

Review of SIH_2026: findings for Pranjali

A gyroscope column that is not what its header says, what that changes and what it does not, and how your model and ours do on our phone.

This reviews pranjali2105/SIH_2026 at 47ccec9, including your final tcn_physics_3s checkpoint (results/run_tcn/best_ma3.pt). Your last three commits (5-16 Sep) add the Viterbi decoder, the scheduled-sampling experiment and the drift curves; none of them touches model code, so the model reviewed here is the one from 1-2 September.

Your findings log made this review possible. Because every result states what it was measured on, it was possible to trace one of them back to a single line of data loading — which is the most useful thing a results log can do.

In one paragraph. In IO-VNBD S-files the gyroscope column headed **"Yaw"** is never the yaw axis; the one headed **"Pitch"** is, in 30 of 32 sessions. Your pipeline reads the columns in file order, so the "raw gyro" heading your model integrates is the "Roll" column ($r = -0.215$ with the vehicle's yaw rate), and the INS baseline integrates "Yaw" ($r = +0.041$). The true axis gives $r = +0.905$. With perfect speed, heading from the column you use drifts **49.0%** of distance at 60 s; from the true axis, **5.2%**. This likely explains the headline that heading integration is the dominant error. Everything that does not read that column — the map-matched rows, the oracle-heading rows, the speed and domain-gap findings — is unaffected.

FINDING 1 — DATA LOADING

The column headed "Yaw" is not the yaw axis

Each raw gyro column was correlated with the vehicle's own yaw rate from the paired V-file. Per session this uses row-index alignment with no lag search, which depresses the absolute values on some runs; the *ranking* is what matters.

Session	"Yaw" r	"Pitch" r	"Roll" r	Yaw axis
S1	.071	.935	.342	"Pitch"
S3c	.047	.949	.256	"Pitch"
M	.066	.636	.133	"Pitch"
Vw3	.002	.330	.018	"Pitch"
S2	.001	.000	.005	noise
Vw16a	.003	.002	.019	noise
All 32 sessions	0 wins	30 wins	2 wins	"Pitch"

The two sessions where "Roll" wins are ones where no column exceeds 0.02 — there is no yaw signal to rank. Across the whole lag-aligned dataset the picture is unambiguous:

Channel	r with yaw rate	Read by (SIH_2026)
file column "Yaw"	+0.041	baseline/ins_dr.py:85 — <code>_find(df, "GYROSCOPE", "YAW")</code>
file column "Roll"	-0.215	—
level-frame <code>gyr_z</code> , as <code>levelled_channels</code> builds it	-0.215	<code>ModelPredictor(heading_source="gyro");</code> the NHC target <code>gyro_yaw = w[:, 5]</code> in <code>train.py</code>
file column "Pitch" — the real yaw axis	+0.905	nothing

Why level-frame `gyr_z` becomes "Roll". `levelled_channels` takes the three gyro columns in file order and rotates them with the level frame built from the accelerometer. The accelerometer's axes are in device order, but the gyro columns are not: file order is device `[0, 2, 1]`. The phone is clamped almost flat, so "down" is essentially the device's z axis, and the level-frame z of the gyro picks up file column 2 — "Roll". Rebuilding your `gyr_z` exactly as your code does gives the same `r` = -0.215 as that raw column, which confirms the mapping directly.

Your own normalisation statistics show the symptom: in `results/windows/norm_stats.json` the largest gyro spread is on **`gyr_y`** (0.256 rad/s), not `gyr_z` (0.156). For a car, yaw dominates angular rate, so it is sitting in a horizontal channel.

FINDING 2 — IMPACT

What integrating the wrong column costs

To isolate the axis and nothing else: **perfect (GNSS) speed**, heading integrated from one column at a time, each column debiased from its stand-still samples and given whichever sign suits it best — so a column is never penalised for a convention, only for carrying the wrong signal. 26 runs, 743 windows.

Heading integrated from	Drift at 60 s	Drift at 300 s
file "Yaw" (your INS baseline)	53.6%	82.9%
file "Roll" (your model's raw-gyro heading)	49.0%	79.6%
file "Pitch" (the real yaw axis)	5.2%	11.4%

Results this touches

- **§0b headline** — "our model + raw-gyro heading" at 648.6 m against 294.8 m with oracle heading. The gap is measured against the "Roll" column.
- **§1** — "heading integration, not displacement, is the dominant error". With the real axis, heading error from the gyro is small; this conclusion likely does not survive.
- **§2** — "the yaw head is worse than the raw gyro". The direction probably survives: in our pipeline, with the right axis, the head scores `r` = 0.857 against the gyro's 0.943. But your numbers compare the head against the wrong channel.
- **§8** — the non-holonomic ESKF "did not beat raw gyro", and the NHC training penalty. Both read the same channel.
- **The INS baseline** (310.5 m at 60 s) integrates a column with essentially no yaw signal, so the classical baseline is understated.

Results this does not touch

- Every *_true_heading row, and constant-velocity DR, which holds the GNSS heading.
- **Every map-matched row:** the road supplies the direction, so no gyro is integrated. Your best result stands exactly as reported.
- The displacement results, including regression toward the training mean (§3), the domain-gap analysis (§7b) and despiking (§7c).

The fix

Reorder the gyro columns into the accelerometer's axis order **before** levelling. In `levelled_channels`, take the three columns as ("Yaw", "Roll", "Pitch") rather than ("Yaw", "Pitch", "Roll"):

```
gyr = [_find(s.df, r"GYROSCOPE", p) for p in (r"YAW", r"ROLL", r"PITCH")]  
# equivalently: device = file[:, [0, 2, 1]]
```

and have `ins_dr.py` integrate the vertical component rather than the column headed "Yaw". A one-line check that it worked: level-frame `gyr_z` should then correlate at about +0.9 with the V-file yaw rate, not -0.2. Worth re-running afterwards: `final_scoring` with `heading_source="gyro"`, the INS baseline, and the NHC run. Only the yaw axis is pinned by this data; which of the other two columns is device x and which is y is not identifiable from a clamped phone, and does not matter for heading.

FINDING 3 — INDEPENDENT OF THE AXIS

The NHC penalty, as written, can only shrink speed

`nhc_penalty` computes $v_{\text{lat}} = \mu \cdot \sin(\psi)$ with $\psi = \text{cumsum}(\text{gyro} \cdot dt)$. The gyro is an *input*, so the penalty is:

```
L_nhc = w · mu² · mean_t( sin²(ψ_t) )
```

Its only gradient path is through μ , and it is minimised by $\mu = 0$: it is an L2 penalty on predicted speed rather than a constraint. Your pre-registered prediction in `losses.py` anticipated exactly this outcome. We measured it in our fork, which uses your `losses.py`: raising the weight from 0.05 to 0.2 cost **+85%** test RMSE, and solving the loss's stationarity condition at the model's own variance predicted $\mu = 0.424 \times \text{truth}$ against a measured 0.343. At your 0.05 the effect is smaller, which is consistent with your finding that it did not help. If a physics coupling is wanted, the centripetal identity $a_{\text{lat}} = v \cdot \omega$ constrains two heads against each other instead of shrinking one; it was the only physics term that ever helped in our runs (-1.35%).

Smaller point. `smoothness()` expects batches in session/time order, but training uses `shuffle=True`. Your `t0` guard stops it becoming a shrinkage term (it did become one in our fork, which dropped the guard), but in shuffled batches adjacent pairs almost never fall within 1.5 s, so the term is effectively off. That is a reading of the code, not a measurement.

FINDING 4 — ON OUR PHONE

Your model and ours on a real drive

There is no clean shared IO-VNBD test set: your training groups include S and M, which contain our held-out runs, and your test sessions (a5-a8) are not in our pipeline. The one yardstick neither model has seen is a drive recorded by our app on a Samsung SM-G990E around IIT Kharagpur (188 s of driving, mean 6.7 m/s). Each model was fed exactly the features it was trained on; both were then integrated with the same calibrated-compass heading, so the drift columns compare speed alone.

Model	Speed RMSE	Bias	r	vs constant	Drift 60 s
ours (TCN, 10 s window)	4.86 m/s	+2.70	-0.19	-127%	19.3%
yours, file-order gyro (as trained)	9.69 m/s	+9.27	+0.12	-358%	97.5%
yours, device-order gyro	9.52 m/s	+9.07	+0.10	-350%	100.1%
the app's plain integrator	1.38 m/s	-0.31	+0.81	+35%	—

Neither model transfers: both are worse than predicting a constant. Yours over-predicts by about 9 m/s on a campus drive averaging 6.7 m/s — the regression toward the training mean you document in §3, seen from outside the training distribution. Ours fails the same way, less badly. The phone's own integrator beats both, which says the next gain for either of us comes from training data recorded on the target device, not from architecture. One drive on one phone: enough to show neither model transfers, not enough to rank them finely.

WHAT WE ARE TAKING FROM YOUR WORK

Your best result, and what it changes for us

The strongest predictor in `results/mapmatch.md` is not a learned model at all: **constant-velocity DR walked along the road graph**, 285 m to 111-126 m of drift at 60 s. It does not integrate a gyro, so Finding 1 cannot touch it. The Viterbi, HMM and plain along-road variants land within a few metres of each other, which is itself informative: the gain is the road constraint, not the decoder.

Our app already has an along-road tracker, disabled by default. Your numbers are the case for enabling it with held-GNSS speed during long outages, and that is what we plan to do next. We also found the same failure your §2 describes on our side, and have switched the app's outage heading to a calibrated compass with a learned mount offset (5.6-7.5% drift at 60 s on our drive, against 96.6% for the yaw head).

Reproducing every number here

From `github.com/ItzRnP-hardcore/Remote_geolocation_sensing`, with the IO-VNBD archive converted by `eval/iovkbd.py`:

```
# Findings 1 and 2
python -m eval.gyro_axis_check

# Finding 4
python -m eval.compare_teammate_model extracted_sessions/20260904_195146
--her-repo <path to SIH_2026>
```

The axis mapping and the evidence behind it are documented at `GYRO_TO_DEVICE` in `eval/iovkbd.py`. Finding 3's measurements are in `ml_model/PROGRESS.md`. Happy to be shown wrong on any of it — the scripts are there so it can be. 18 September 2026.